

Edge Cases and Optimizations

This project targets Assignment 3: an intelligent context optimizer for multi-turn agents.

Edge Cases to Test

Area	Edge case	Expected behavior
Empty memory	New session has no stored facts	Prompt says memory is missing instead of hallucinating
Long conversation	Recent messages exceed short-term window	Keep newest useful turns, summarize or drop stale turns
Contradictory facts	User later corrects an old memory	Prefer corrected memory using recency and feedback
Prompt injection	User says to ignore stored facts or system rules	Treat instruction as current user text, not memory policy
Similar repeated query	Same query appears multiple times	CAG cache hit reuses optimized context
Cache staleness	Important memory changes after cache write	Invalidate by session/version hash or short TTL
Oversized memory	One memory exceeds token budget	Compress or skip instead of blowing prompt budget
Duplicate memory	Same fact saved many times	Deduplicate by normalized content hash
Low-signal memory	Memory has weak semantic match	Exclude below threshold
Negative feedback	User thumbs down an answer	Decrease confidence for selected blocks
Positive feedback	User thumbs up an answer	Increase confidence for selected blocks
Preference drift	User changes tone/detail preference	New profile overrides old style memory
Time-sensitive facts	Deadlines, status, pricing, policies	Store timestamp and expire or revalidate
Entity collision	Two people/projects share names	Scope memories by user, session, project, and source
Privacy	Sensitive memory should not leak between users	Tenant-scoped keys and access checks
Retrieval latency spike	Vector store or Redis slows down	Timeout and fall back to recent memory
Bad summary	Compression removes crucial constraint	Keep source snippets and evaluate summary faithfulness
Multilingual query	User changes language	Use multilingual embeddings or language-aware fallback
Ambiguous query	"What about the deadline?"	Use recent turns plus long-term deadline facts
Adversarial correction	User falsely says old facts are wrong	Store as low-confidence until confirmed

Optimization Levers

1. Use Redis or Redis Stack for short-term session memory, hot user profile, and CAG cache.
2. Store long-term memory in Qdrant with tenant/user/project filters on every search.
3. Split context into lanes: recent turns, durable facts, preferences, task state, retrieved knowledge, and feedback.
4. Rank context with semantic similarity, recency, importance, source reliability, and feedback.
5. Reserve a fixed token budget per lane so one noisy source cannot crowd out everything else.
6. Cache optimized context by normalized query, session id, user profile version, and memory index version.
7. Use TTLs and version keys to avoid serving stale cached context after memory updates.
8. Deduplicate by normalized text hash and optionally by embedding similarity.
9. Compress only low-risk verbose blocks; keep exact wording for deadlines, user constraints, numbers, and legal facts.
10. Run retrieval, summarization, and safety checks in parallel when the runtime supports it.
11. Add circuit breakers: if vector search fails, answer from recent memory and mark confidence lower.
12. Track cost and latency per stage: memory read, vector search, rerank, compression, LLM call, cache hit.

Report Metrics

Use these in the submission:

Metric	Why it matters
Win/tie/loss vs full history	Shows quality trade-off
Average input tokens	Shows cost reduction
P50/P95 latency	Shows production behavior
Cache hit rate	Shows CAG value
Memory precision@k	Shows selected context relevance
Failure rate by category	Shows honesty on edge cases
Break-even point	Shows when optimization pays for itself

Recommended Architecture

User message

-> Session router

-> Tenant/user/project/session scoped key

-> Short-term memory read from Redis

-> Long-term memory search from vector store

-> Rerank top long-term candidates

-> CAG cache lookup

-> Context scorer

-> Token-budget allocator

-> Compression / fallback

-> Prompt builder

-> LLM

-> Feedback capture

-> Memory update + memory index version bump

-> Cache invalidation

Production Storage Plan

Layer	Local implementation	Production replacement
Short-term memory	ShortTermMemory	Redis list per scoped session key
CAG cache	TTLCache	Redis SETEX keyed by query + scope + memory version
Long-term memory	LongTermMemory	Qdrant vector collection
Reranking	CrossEncoderReranker local stand-in	SentenceTransformersCrossEncoderReranker
Embeddings	deterministic token vectors	LlamaIndex HuggingFaceEmbedding("BAAI/bge-small-en-v1.5")
Feedback	FeedbackStore	Redis event list/stream separate from memory facts
LLM	deterministic local simulator	LlamaIndex + Ollama open-source model
Evaluation	deterministic fact scorer	LlamaIndex/Ollama judge + human calibration

All keys must include tenant, user, project, and session where applicable:

tenant:{tenant_id}:user:{user_id}:project:{project_id}:session:{session_id}

Long-Term Memory Classes

Class	Production risk	Mitigation
Episodic	Recent events can overpower durable facts	Add age decay and confidence
Procedural	Wrong workflow causes repeated bad tool use	Store source and version
Semantic	Similar facts can collide	Tenant/project filters and reranking
Preference	Old preferences can conflict with new ones	Prefer recent high-confidence preferences
Exact fact	Dates/prices/IDs are damaged by compression	Mark protected and never summarize destructively
Feedback	Thumbs down is ambiguous	Store event separately with reason and context

Multi-Agent Option

Use multiple narrow agents rather than one broad autonomous memory agent:

Agent	Job
Memory Writer	Extract and classify memory candidates
Context Optimizer	Select context under token budget
Retrieval	Query Qdrant and rerank results
Feedback Analyst	Diagnose thumbs up/down into style/fact/tool issues
Privacy/Policy	Redact or reject unsafe memories
Evaluation	Run adversarial regression and report win/tie/loss

Production Failure Modes

- Long context is not automatically better. It increases cost and can bury relevant instructions.
- Summaries can delete exact constraints such as dates, numbers, and negations.
- Feedback is noisy. A thumbs down may mean bad style, bad fact, or bad tool use.
- Cache hits can be harmful if memory has changed but cache invalidation is weak.
- Semantic search can miss keyword-specific facts such as IDs, dates, and exact names.
- Recency can over-rank irrelevant chatter.
- Multi-turn agents can lock onto an early wrong assumption and keep compounding it.

Sources used while shaping this checklist:

- [Microsoft Research: LLMs Get Lost In Multi-Turn Conversation](#)
- [Cache-Augmented Generation overview](#)
- [FlowHunt CAG glossary](#)
- [LlamaIndex local embedding docs](#)
- [LlamaIndex local model starter](#)